PremadeGG

Valorant LFG Platform

Guion de Funcionamiento

Descripción completa de la arquitectura,
funcionalidades y flujos de la aplicación

21 de febrero de 2026

Índice

1. ¿Qué es PremadeGG? 3

2. Stack Tecnológico (MEAN) 3

3. Arquitectura del Proyecto 3

4. Modelos de Datos 4

4.1. User (Usuario) . . . 4

4.2. Team (Equipo) . . . 4

4.3. LFGPost (Publicación LFG) . . . 5

4.4. Reputation (Review / Karma) . . . 5

4.5. Conversation y Message . . . 5

4.6. Notification . . . 5

5. Flujos de Funcionamiento 5

5.1. Registro y Login . . . 6

5.2. Recuperación de Contraseña . . . 6

5.3. Explorar Jugadores . . . 6

5.4. Sistema de Karma (Reputación) . . . 6

5.5. Tablón LFG (Looking For Group) . . . 7

5.6. Equipos . . . 7

5.7. Mensajería en Tiempo Real . . . 7

5.8. Notificaciones . . . 7

6. Integración con Riot Games API 8

7. Frontend — Páginas y Navegación 8

7.1. Componentes Compartidos . . . 8

8. Servicios del Frontend 8

9. Seguridad **9**

10. Despliegue **10**

10.1. Docker (Desarrollo Local) 10

11. Resumen del Flujo Completo **11**

1. ¿Qué es PremadeGG?

PremadeGG es una plataforma **"Looking For Group" (LFG)** para jugadores de **Valorant**. Su objetivo es conectar jugadores basándose no solo en su rango competitivo, sino también en su **reputación (Karma)** y compatibilidad de roles.

⚠ Sistema Anti-Toxicidad

Los jugadores con **Karma inferior a 3** son **filtrados del matchmaking** principal, fomentando una comunidad sana y libre de toxicidad.

2. Stack Tecnológico (MEAN)

La aplicación utiliza el stack **MEAN**, complementado con servicios externos:

Capa	Tecnología	Función
Base de datos	MongoDB + Mongoose	Almacena usuarios, equipos, posts, mensajes, notificaciones y reviews
Backend	Express.js + Node.js	API REST con arquitectura MVC estricta
Frontend	Angular (Standalone)	SPA con SASS 7-1, modo oscuro y glassmorphism
Tiempo real	Socket.IO	Notificaciones y mensajería instantánea
Email	Brevo (Sendinblue)	Envío de correos de recuperación de contraseña
API externa	Riot Games API (Henrik Dev)	Validación de cuentas y obtención de rango real

Cuadro 1: Tecnologías utilizadas en PremadeGG

3. Arquitectura del Proyecto

El proyecto se organiza como un **monorepo** con dos carpetas principales:

```
premade-gg/
|-- server/                    <- Backend (Express MVC)
```

```
|   |-- app.js                  <- Punto de entrada
|   |-- config/db.js            <- Conexion a MongoDB
|   |-- models/                 <- 7 modelos Mongoose
|   |-- controllers/            <- 9 controladores
|   |-- routes/                 <- 9 ficheros de rutas
|   |-- middlewares/            <- Auth (JWT) y manejo de errores
|   |-- services/               <- Email, Riot API, Socket.IO
|   `-- utils/seedDefaults.js   <- Datos de prueba
|
|-- client/                     <- Frontend (Angular)
|   `-- src/app/
|       |-- pages/              <- 13 paginas
|       |-- shared/             <- 8 componentes reutilizables
|       |-- services/           <- 5 servicios
|       |-- guards/             <- Guard de autenticacion
|       |-- interceptors/       <- Interceptor HTTP (JWT)
|       `-- models/             <- 8 interfaces TypeScript
|
`-- docker-compose.yml          <- Orquestacion de contenedores
```

4. Modelos de Datos

4.1 User (Usuario)

Almacena toda la información del jugador:

- **Autenticación:** email, contraseña (hasheada con bcrypt), tokens de reset.
- **Identidad Riot:** `riotId` (ej: `Player#1234`), `puuid`, región (`eu`, `na`, `ap`, `kr`).
- **Datos de juego:** rango (`Iron` → `Radiant`), rol (`Duelist`, `Controller`, `Initiator`, `Sentinel`, `Flex`), horario, agentes favoritos (máx. 3).
- **Karma:** puntuación de 0 a 5 (por defecto 5), calculada como media de las reviews recibidas.
- **Perfil:** bio, imagen de perfil, redes sociales (`Discord`, `Twitter`, `Twitch`), disponibilidad por día.
- **Flags:** `lookingForTeam`, `isCoach`.

4.2 Team (Equipo)

- **Identidad:** nombre (único, máx. 40 char), tag (máx. 6 char), logo.

- **Personas:** `owner` (creador), `coach`, `players[]` (con rol asignado), `staff[]` (manager, analyst, content_creator, streamer).
- **Configuración:** región, rango objetivo, redes sociales, logros.
- **Reclutamiento:** flag `recruiting`, roles que buscan, detalles.
- **Solicitudes de ingreso:** `joinRequests[]` con estado (`pending`, `accepted`, `rejected`).

4.3 LFGPost (Publicación LFG)

- **Datos:** autor, título, descripción, tamaño del equipo (2, 3 o 5), rango mínimo, karma mínimo (default 3).
- **Miembros:** array con usuario y estado (`pending`/`accepted`).
- **Estado:** `open`, `full`, `closed`.
- **TTL:** expira automáticamente a las 24 horas (MongoDB borra el documento vía TTL index).

4.4 Reputation (Review / Karma)

- **Relación:** `author` → `target` (un usuario valora a otro).
- **Datos:** rating (1–5), comentario, flag `playedTogether`.
- **Restricción:** un author solo puede dejar una review a cada target (índice único compuesto).

4.5 Conversation y Message

Soporte para chats **directos** y **grupales**. Cada conversación tiene participantes, último mensaje y contadores de mensajes no leídos.

4.6 Notification

Notificaciones in-app (solicitudes de equipo, reviews, mensajes) con estado leído/no leído. Se entregan en tiempo real vía **Socket.IO**.

5. Flujos de Funcionamiento

5.1 Registro y Login

1. El usuario se registra con email, contraseña y Riot ID (`POST /api/auth/register`).
2. Se **valida el Riot ID** contra la API de Riot Games (Henrik Dev).
3. Se obtiene el **rango real** del jugador automáticamente.
4. Se devuelve un **token JWT**.
5. Para login posterior: `POST /api/auth/login` → verifica credenciales → devuelve JWT.

Autenticación JWT

El JWT se incluye en todas las peticiones autenticadas vía `Authorization: Bearer <token>`. El frontend tiene un **interceptor HTTP** que lo inyecta automáticamente. Un **auth guard** protege las rutas que requieren login.

5.2 Recuperación de Contraseña

1. `POST /api/auth/forgot-password` → genera token temporal.
2. Se envía un email con enlace de recuperación usando **Brevo**.
3. `POST /api/auth/reset-password` → verifica token y actualiza contraseña.

5.3 Explorar Jugadores

- Página `/players`: lista usuarios con paginación por cursor (scroll infinito).
- Búsqueda por Riot ID (`/api/users/search?q=...`).
- Cada jugador se muestra con una **Player Card**: nombre, rango, rol, karma y agentes favoritos.
- Los jugadores con Karma < 3 son **filtrados por defecto**.

5.4 Sistema de Karma (Reputación)

1. Un usuario deja una review (1–5 estrellas + comentario) a otro usuario (`POST /api/reviews`).
2. Se **recalcula el karma** del usuario valorado como media de todas sus reviews.
3. Si el karma baja de 3 → el usuario queda **filtrado del matchmaking**.
4. Solo se permite **una review por par** author–target (índice único).
5. Endpoint de elegibilidad: se verifica antes de mostrar el formulario al usuario.

5.5 Tablón LFG (Looking For Group)

1. Un usuario crea un post LFG con: título, descripción, tamaño (2/3/5), rango mínimo, karma mínimo (`POST /api/lfg`).
2. El post aparece en el tablón `/lfg` con estado `open`.
3. Otros usuarios solicitan unirse (`POST /api/lfg/:id/join`) → estado `pending`.
4. El autor acepta/rechaza vía `PUT /api/lfg/:id/members/:userId`.
5. Cuando se llena → estado `full`.
6. A las **24 horas** → MongoDB lo borra automáticamente (TTL index).

5.6 Equipos

1. Un usuario crea un equipo con nombre, tag, logo, región y rango objetivo (`POST /api/teams`).
2. Se convierte en **owner** del equipo.
3. Otros usuarios solicitan unirse (`POST /api/teams/:id/requests`) → estado `pending`.
4. El owner acepta/rechaza vía `PUT /api/teams/:id/requests/:userId`.
5. Si acepta → se añade a `players[]` con su rol.
6. El owner puede: asignar coach, añadir staff, activar reclutamiento, eliminar equipo (cascada).

5.7 Mensajería en Tiempo Real

1. Un usuario crea un chat directo con otro (`POST /api/messages/conversations/direct`).
2. Envía un mensaje (`POST /api/messages/conversations/:id/messages`).
3. **Socket.IO** entrega el mensaje en **tiempo real** si el destinatario está conectado.
4. Se actualizan los contadores de no leídos automáticamente.
5. También soporta **conversaciones grupales**.

5.8 Notificaciones

Se crean automáticamente cuando alguien solicita unirse a un equipo, se recibe una review, etc. Se muestran en un **icono de campana** en la navbar con contador de no leídas. Se entregan en tiempo real vía **Socket.IO**.

6. Integración con Riot Games API

La aplicación se integra con la API de Riot Games a través del servicio **Henrik Dev API**:

Endpoint	Función
`GET /api/riot/validate/:gameName/:tagLine`	Valida que una cuenta de Riot existe realmente
`GET /api/riot/rank/:region/:gameName/:tagLine`	Obtiene el rango competitivo actual del jugador

Cuadro 2: Endpoints de integración con Riot API

Esto garantiza que:

- Los usuarios **no pueden inventar** un Riot ID falso.
- Los rangos mostrados son **reales y verificados**.

7. Frontend — Páginas y Navegación

7.1 Componentes Compartidos

- **Navbar:** navegación principal con campana de notificaciones y avatar.
- **PlayerCard:** tarjeta de jugador con rango, karma y rol.
- **TeamCard:** tarjeta de equipo con logo, tag y miembros.
- **LFGCard:** tarjeta de post LFG con información resumida.
- **KarmaBadge:** indicador visual del karma (color según nivel).
- **RankBadge:** badge visual del rango competitivo.
- **NotificationBell:** campana con contador de no leídas.

8. Servicios del Frontend

Ruta	Página	Auth	Descripción
`/`	Home	—	Landing page de la plataforma
`/players`	Players	—	Listado con búsqueda y filtros
`/lfg`	LFG Board	—	Tablón de publicaciones LFG
`/lfg/:id`	LFG Detail	—	Detalle de un post LFG
`/teams`	Teams	—	Listado de equipos
`/teams/:id`	Team Detail	—	Detalle de un equipo
`/profile/:id`	Profile	—	Perfil público de un jugador
`/settings/profile`	Profile Edit	🔒	Editar perfil propio
`/messages`	Messages	🔒	Bandeja de mensajes
`/login`	Login	—	Inicio de sesión
`/register`	Register	—	Registro
`/forgot-password`	Forgot Password	—	Recuperar contraseña
`/reset-password`	Reset Password	—	Restablecer contraseña

Cuadro 3: Páginas del frontend (🔒 = requiere autenticación)

Servicio	Responsabilidad
`AuthService`	Login, registro, gestión de JWT, estado de autenticación
`ApiService`	Todas las llamadas HTTP a la API (con caché in-memory de 60s)
`MessageService`	Gestión de conversaciones y mensajes
`NotificationService`	Polling/socket de notificaciones, contadores
`SocketService`	Conexión WebSocket con Socket.IO

Cuadro 4: Servicios Angular del frontend

9. Seguridad

- **JWT:** autenticación stateless con tokens firmados.
- **Bcrypt:** contraseñas hasheadas antes de almacenarse.
- **CORS:** en producción, solo acepta peticiones del dominio del frontend (Vercel).
- **Validación de datos:** Mongoose schemas con validaciones estrictas (longitud, enums, unicidad).
- **Middleware de errores:** captura y formatea todos los errores de forma consistente.

- **Logs condicionales:** Morgan en modo `dev` (desarrollo) o solo errores (producción).

10. Despliegue

Componente	Plataforma	Detalle
Frontend	Vercel	`premade-gg.vercel.app`
Backend	Render	Configurado vía `render.yaml`
Base de datos	MongoDB Atlas	Cluster en la nube

Cuadro 5: Plataformas de despliegue

10.1 Docker (Desarrollo Local)

```
docker-compose up --build          # Levanta API + MongoDB + Mongo Express
docker-compose exec api npm run seed   # Poblar datos de prueba
```

Servicios disponibles en local:

- **API:** `http://localhost:4000`
- **Mongo Express** (UI para inspeccionar BD): `http://localhost:8082`
- **MongoDB:** `localhost:27017`

11. Resumen del Flujo Completo

Ciclo de vida de un usuario en PremadeGG

1. **Registro:** se registra → valida Riot ID → obtiene rango real.
2. **Exploración:** explora jugadores → filtra por rango, rol y karma.
3. **LFG:** crea o se une a posts LFG → encuentra grupo para jugar.
4. **Equipos:** crea o se une a un equipo → estructura competitiva.
5. **Reputación:** deja reviews → construye karma en la comunidad.
6. **Comunicación:** chatea en tiempo real → coordina con compañeros.
7. **Notificaciones:** recibe alertas → solicitudes, reviews, mensajes.

⚠ Karma <3 → Filtrado del matchmaking (anti-toxicidad)